
1.Student Details.html

- **Main Feature:** Displays a static, well-styled table of student information.
- **HTML Attributes & Elements:**
 - `<table>`: The main container for the table.
 - `<thead>` & `<tbody>`: Sections for the table header and table body, respectively.
 - `<tr>`: Defines a table row.
 - `<th>`: Defines a header cell (bold, centered).
 - `<td>`: Defines a data cell.
 - `<div class="container">`: A generic container used to center the table and give it a "card" look.
 - `<link href="...">`: Imports the 'Roboto' font from Google Fonts.
- **CSS Properties & Concepts:**
 - `box-shadow`: Adds a soft shadow to the `.container` to make it "float."
 - `border-radius`: Gives the container rounded corners.
 - `border-collapse`: Makes the table borders look like a single, clean line.
 - `tr:nth-child(even)`: A **pseudo-class** that applies a background color to even-numbered rows (this is called "zebra-striping").
 - `tr:hover`: A **pseudo-class** that changes the background color of a row when you mouse over it.
- **JavaScript:** None. This is a static page.

2.Face Detection.html

- **Main Feature:** Uses a pre-trained Artificial Intelligence (AI) model to find and draw boxes around human faces in an image uploaded by the user.
- **Key Technologies (External Libraries):**

- **TensorFlow.js (@tensorflow/tfjs)**: A library that allows you to run machine learning models in the browser.
 - **BlazeFace (@tensorflow-models/blazeface)**: The specific, pre-trained AI model that is fast and optimized for detecting faces.
 - **HTML Attributes & Elements:**
 - `<input type="file">`: A special input that lets the user select a file from their device.
 - `<canvas>`: A "drawing board" element. It's used to first display the uploaded image and then draw the red rectangles on top.
 - **JavaScript Attributes & Concepts:**
 - `blazeface.load()`: An **asynchronous** function (using `.then()`) that loads the AI model.
 - `addEventListener("change", ...)`: A listener that runs a function when the user uploads a file.
 - `URL.createObjectURL()`: Creates a temporary URL for the uploaded image file.
 - `canvas.getContext("2d")`: Gets the 2D drawing tools for the canvas.
 - `ctx.drawImage()`: A canvas function to draw an image.
 - `await model.estimateFaces(canvas)`: The main AI function. It "estimates" where the faces are on the canvas and returns an array of coordinates.
 - `forEach()`: A loop used to draw a rectangle for each face found.
 - `ctx.strokeRect()`: A canvas function that draws the outline of a rectangle.
-

5.Sentiment Analysis.html

- **Main Feature:** Analyzes a block of text and classifies it as "Positive," "Negative," or "Neutral" based on keywords.
- **HTML Attributes & Elements:**
 - `<textarea>`: A multi-line text box for user input.
 - `<button>`: The button that triggers the analysis.

- `<div id="result">`: An empty container that will be filled with the result.
 - **CSS Properties & Concepts:**
 - `.positive`, `.negative`, `.neutral`: CSS classes that are dynamically added to the result div to change its color (green, red, or gray).
 - **JavaScript Attributes & Concepts:**
 - `addEventListener('click', ...)`: Runs the `analyzeSentiment` function when the button is clicked.
 - `document.getElementById()`: The main way to select and read from (or write to) HTML elements.
 - `.value.toLowerCase().trim()`: A common chain: gets the text, converts it to lowercase (for easier matching), and removes extra whitespace.
 - `const positiveWords = [...]`: Uses **Arrays** to store lists of keywords.
 - `text.includes(word)`: A simple string method that checks if a string contains a specific word.
 - `score++` / `score--`: A simple scoring system to track sentiment.
 - `resultDiv.textContent = ...` & `resultDiv.className = ...`: This is **DOM Manipulation**. The script changes the text and the CSS class of the result div to show the answer.
-

6.Spam Detection.html

- **Main Feature:** A more advanced text analyzer that checks for "spam" keywords. It uses simple **Natural Language Processing (NLP)** techniques.
- **HTML Attributes & Elements:**
 - `<textarea>`, `<button>`, `<div id="result">`: Same as the Sentiment Analysis tool.
- **JavaScript Attributes & Concepts:**
 - `const spamWords = [...]` & `const stopwords = [...]`: "Stopwords" are common words (like "is", "the", "a") that are filtered out to reduce noise.
 - **Regular Expressions (RegEx):**

- function stem(word): Uses a RegEx (/ing\$|ed\$s\$/) to remove common word endings. This is called "**stemming**" (e.g., "winning" becomes "win").
 - function cleanText(text): Uses RegEx to remove URLs and punctuation, cleaning the text for analysis.
 - **Processing Pipeline:** The preprocess(text) function is a pipeline. It takes raw text and, in order:
 1. Makes it lowercase.
 2. Cleans it (removes URLs/symbols).
 3. Splits it into an array of words (.split()).
 4. Removes stopwords (.filter()).
 5. Stems each word (.map()).
 - checkSpam(msg): The main logic function that uses the processed "tokens" (words) to calculate a spam score.
-

7.Text to Speech.html

- **Main Feature:** Converts any text the user types into audible, computer-generated speech.
- **Key Technologies (Browser API):**
 - **Web Speech API:** A feature built directly into modern browsers.
- **HTML Attributes & Elements:**
 - <textarea>, <button onclick="...">: A simple text area and a button.
 - onclick="speakText()": An **inline event handler** attribute. It directly calls the speakText() function when the button is clicked.
- **JavaScript Attributes & Concepts:**
 - new SpeechSynthesisUtterance(text): This is the core of the API. It creates a "speech request" object containing the text you want to speak.
 - speech.rate, speech.pitch, speech.volume: These are **properties** of the speech request. You can change them to make the voice faster/slower, higher/lower, or louder/softer.

- `window.speechSynthesis.speak(speech)`: This is the command that tells the browser's speech "engine" to take your speech request and say it out loud.
-

8.Movie Recommender.html

- **Main Feature:** Recommends movies from a hard-coded database based on a shared genre.
 - **HTML Attributes & Elements:**
 - `<select>`: Creates a dropdown menu.
 - `<option value="...">`: An item within the dropdown. The value attribute is what JavaScript reads.
 - `<span id="liked">` & `<span id="result">`: Empty `<span>` tags that act as placeholders. JavaScript will fill them with text.
 - **JavaScript Attributes & Concepts:**
 - `const movieGenres = {...}`: An **Object** is used as a simple "key-value" database. The key is the movie title, and the value is an array of its genres.
 - `document.getElementById("movieSelect").value`: Gets the value of the currently selected `<option>`.
 - `myGenres = movieGenres[movie]`: Accessing the value (the genre array) from an object using its key (the movie title).
 - `for (const title in movieGenres)`: A **for...in loop**, which is used to iterate over the *keys* (the movie titles) of an object.
 - `Array.some(g => ...)`: A powerful array method. It checks if **at least one** item in an array passes a test. Here, it checks if any genre in one movie is also in the other movie's genre list.
 - `document.getElementById("result").textContent = ...`: **DOM Manipulation** to write the final list of recommendations into the `<span>`.
-

9.Secure Login.html

- **Main Feature:** A login form that performs **client-side validation** (checking the data in the browser) before it's "submitted." It checks for password strength and blocks basic security risks.
- **HTML Attributes & Elements:**
 - `<form id="loginForm">`: A container for inputs.
 - `<input type="text">`, `<input type="password">`: Note the `type="password"` which automatically hides the characters.
 - `<button type="submit">`: A special button. When inside a `<form>`, clicking it triggers the form's "submit" event.
- **CSS Properties & Concepts:**
 - `.error` & `.success`: Classes used to color the feedback messages red or green.
- **JavaScript Attributes & Concepts:**
 - `form.addEventListener('submit', ...)`: This is the standard way to handle form submissions.
 - `e.preventDefault()`: This is **crucial**. It stops the form's default behavior (which is to reload the page) so the script can perform its checks.
 - **Regular Expressions (RegExp)**: This is the core feature.
 - `usernamePattern`: Checks for a valid username format.
 - `passwordPattern`: A complex RegExp that uses "positive lookaheads" (`?=...`) to enforce rules: must contain one lowercase, one uppercase, one number, one special character, and be 8+ characters long.
 - `sqlPattern`: A simple "blocklist" RegExp that looks for keywords used in **SQL Injection attacks** (like `SELECT`, `DROP`, etc.).
 - `pattern.test(username)`: This is the method that runs the RegExp check against a string. It returns true or false.

10.ShopSmart.html

- **Main Feature:** A mock e-commerce page that shows "recommended" products from the same category when a user clicks an item.

- **HTML Attributes & Elements:**
 - `onclick="viewProduct(...)":` The function is called with **parameters** ('T-Shirt', 'clothing') passed directly from the HTML.
- **CSS Properties & Concepts:**
 - `display: flex:` The main property for the **Flexbox** layout model.
 - `flex-wrap: wrap:` Allows the product items to wrap to the next line on smaller screens.
 - `gap: 15px:` A modern CSS property to add space between flex items.
 - `.product:hover { transform: scale(1.05); }:` A CSS **transform** that makes the product "grow" slightly on mouse hover,
- **JavaScript Attributes & Concepts:**
 - `const products = [...]:` An **Array of Objects**. This is a common and powerful way to store structured data. Each object is a product with its own properties.
 - `Array.filter(p => p.category === category):` The **filter()** method. It creates a new array (related) containing *only* the products from the original products array that match the selected category.
 - `Array.map(p => ...):` The **map()** method. It loops through the related array and *transforms* each product object into an HTML string.
 - `Array.join(""):` Joins all the individual HTML strings from the `.map()` result into one single block of HTML.
 - `container.innerHTML = ...:` **DOM Manipulation**. It takes the new block of HTML and "injects" it into the page, replacing "No recommendations yet."

3 & 4.High-Level Overview

- **login.html (Front-end):** A simple HTML page that shows the user a "Login" form.
- **register.html (Front-end):** A simple HTML page that shows the user a "Register" form.
- **server.js (Back-end):** This is the **brain** of the application. It's a server that:
 1. Shows the login.html and register.html pages to the user.

2. Receives the data when a user clicks "Login" or "Register".
 3. Connects to a **MongoDB database** to save or look up user information.
 4. Uses **password encryption (bcrypt)** to securely store passwords.
 5. Uses **sessions (express-session)** to keep a user logged in.
-

1. login.html

- **Main Feature:** This is the **login page** for the user. It's a simple, static HTML form.
 - **HTML Attributes & Elements:**
 - `<form action="/login" method="POST">`: This is the most important part.
 - `action="/login"`: Tells the browser to send the form's data to the `/login` path on the server.
 - `method="POST"`: Specifies *how* to send the data. POST is used for sending sensitive information (like passwords) securely.
 - `<input type="email" name="email">`: A text field for the email. The `name="email"` attribute is used by the server to identify this piece of data.
 - `<input type="password" name="password">`: A password field. The `name="password"` attribute identifies this data.
 - `<a href="/register.html">Register</a>`: A simple link that navigates the user to the registration page.
 - **JavaScript:** None. This is a "dumb" HTML page; all the logic is handled by the `server.js` back-end.
-

2. register.html

- **Main Feature:** This is the **registration page** (or "Sign Up" page). It's almost identical to `login.html` but collects one extra piece of information.
- **HTML Attributes & Elements:**
 - `<form action="/register" method="POST">`:

- `action="/register"`: Tells the browser to send this form's data to the `/register` path on the server.
 - `<input type="text" name="username">`: An additional field to get the user's name.
 - `<input type="email" name="email">`: Same as the login form.
 - `<input type="password" name="password">`: Same as the login form.
 - **JavaScript**: None. Like the login page, all logic is handled by the `server.js` back-end.
-

3. `server.js` (The Back-End)

This is the "brain" that makes the other two files work. It's a **Node.js** server.

- **Main Feature**: It handles all user logic: creating new accounts, logging in existing users, securing passwords, and managing who is "logged in."
- **Key Technologies (Libraries)**:
 - **express**: A framework for building web servers in Node.js. It's used to create the different paths (like `/login`, `/register`, `/`).
 - **mongoose**: A library that makes it easy to talk to a **MongoDB database**. It's used to save and find user data.
 - **bcrypt**: A crucial **security library** used for password hashing. It scrambles passwords (e.g., "1234" becomes `2b$10$...`) so that even if the database is stolen, the passwords are not readable.
 - **express-session**: A library that manages **sessions**. This is how the server "remembers" who a user is. When you log in, the server gives you a "cookie," and this library checks for that cookie on future requests.
- **Key Attributes & Concepts**:
 - `app.post("/register", ...)`: This block of code **listens for data** sent from the `register.html` form. It does the following:
 1. Checks if the email already exists (`User.findOne`).
 2. **Hashes** the password using `bcrypt.hash`.
 3. Saves the new user (with the *hashed* password) to the database.

- `app.post("/login", ...)`: This block **listens for data** from the login.html form.
 1. Finds the user by their email (`User.findOne`).
 2. Compares the password from the form with the *hashed* password in the database using `bcrypt.compare`.
 3. If they match, it saves the user's data to the **session** (`req.session.user = user`), officially "logging them in."
- `function auth(req, res, next)`: This is **middleware**. It's a security guard. It checks if a user is logged in (`req.session.user`) before letting them access a protected page (like the main / page).
- `app.get("/logout", ...)`: A path that destroys the session (`req.session.destroy()`) and logs the user out.

🔍 **HTML**: HyperText Markup Language

🔍 **CSS**: Cascading Style Sheets

🔍 **JS**: JavaScript

🔍 **WWW**: World Wide Web

🔍 **HTTP**: Hypertext Transfer Protocol

🔍 **HTTPS**: Hypertext Transfer Protocol Secure

🔍 **URL**: Uniform Resource Locator

🔍 **URI**: Uniform Resource Identifier

🔍 **API**: Application Programming Interface (Used in your Face Detection & Text-to-Speech files)

🔍 **DOM**: Document Object Model (This is what your JavaScript manipulates)

🔍 **AJAX**: Asynchronous JavaScript and XML (A technique for updating a page without reloading)

🔍 **JSON**: JavaScript Object Notation (A popular format for sending data between a server and a web page)

🔍 **SQL**: Structured Query Language (The language used to talk to databases)

❓ **CRUD:** Create, Read, Update, Delete (These are the four basic operations of any database system)

❓ **MERN:** MongoDB, Express, React, Node.js (A popular "stack" or set of technologies to build a full app. Your server.js file uses **M**ongoDB, **E**xpress, and **N**ode.js)